

Pseudocode

```
PROMPT for number of students
GET number_students          A

ASSERT 0 ≤ number_students
FOR id ← 1 ... number_students
    student_present[id] ← FALSE    B

done ← FALSE
WHILE more students
    PROMPT for student ID
    GET id                        C

    IF 1 ≤ id ≤ number_students
        ASSERT 0 ≤ id ≤ size(student_present)
        student_present[id] ← TRUE    D
    ELSE
        PUT value is out of range
        done ← TRUE                E

FOR id ← 1 ... number_students    F
    IF student_present[id] == FALSE
        PUT that student id is absent    G
```

Quality

Algorithmic efficiency:

- A. $O(1)$ Only executed once
- B. $O(n)$ One iteration for every student in the class
- C. $O(n)$ Each iteration is $O(1)$ but gets executed n times
- D. $O(n)$ Once for each student in attendance
- E. $O(n)$ Once for each student in attendance
- F. $O(1)$ This is our exit condition.
- G. $O(n)$ once for student in class.
- H. $O(m)$ where there are m students who are missing. Probably 1.

Understandability

Obvious if comments are added, straightforward as it currently stands in pseudocode

- `number_students`: Clearly the number of students
- `student_present`: Perhaps could be better named. An array of Booleans indicating if a student is present.
- `id`: clearly a student ID

Malleability

The number of students in the file is given by the user, making for configurable malleability.

Quality

Asserts

Asserts verify the indices are within range.

Trace

	number_students	student_present	id	done	output
A	3	/	/	/	
B	3	[?, f, ?, ?]	1	/	
B	3	[?, f, f, ?]	1	/	
B	3	[?, f, f, f]	1	/	
C	3	[?, f, f, f]	2	f	
D	3	[?, f, t, f]	2	f	
C	3	[?, f, t, f]	1	f	
D	3	[?, t, t, f]	1	f	
C	3	[?, t, t, f]	0	f	
E	3	[?, t, t, f]	0	t	
F	3	[?, t, t, f]	1	t	
G	3	[?, t, t, f]	1	t	
F	3	[?, t, t, f]	2	t	
F	3	[?, t, t, f]	3	t	1